Fase 2 — Análise dos requisitos não funcionais

Grupo PL3-G7 · RAS 2026/27 · 11 de novembro de 2026

1 Resumo

Os dois RNF propostos para a Fase 2 são **eficiência de desempenho** da ISO/IEC 25010, com duas subcaracterísticas distintas:

RNF	Subcaracterística ISO/IEC 25010	Problema em uma frase
RNF-01 Carregamento de ficheiros grandes	Utilização de recursos (memória) e comportamento temporal	Um vídeo de até 1 GB é guardado inteiro em memória em três serviços seguidos.
RNF-02 Elasticidade do processamento de vídeo	Capacidade (a ISO 25010:2023 também lista a escalabilidade, em Flexibilidade)	Um único `video_tool` processa um vídeo de cada vez, e as pré-visualizações esperam atrás dos processamentos completos.

A medição exploratória confirma os dois problemas. No RNF-01, um único salto que bufferiza chega a cerca de 3 GB de RAM para um ficheiro de 1 GB. Com streaming, o mesmo salto fica em cerca de 90 MB. No RNF-02, com 4 processamentos em fila, a pré-visualização só começa ao fim de 14,7 s. Com 2 consumidores na mesma máquina de 2 núcleos o tempo total não melhora. Por isso, a elasticidade tem de vir de réplicas com recursos próprios e de uma fila separada para pré-visualizações, e não apenas de mais threads.

O ponto de partida é o commit oficial `052310226c51fa57a17101c7bd1ad795674aac95` do repositório da solução PL1-G4. As medições exploratórias foram feitas sem Docker, num ambiente com 2 vCPU e 8 GB de RAM. As medições oficiais de base devem ser repetidas no Docker de um dos portáteis do grupo, com os scripts da pasta `medicoes/`.

2 RNF-01 — Diagnóstico do carregamento de ficheiros grandes

O vídeo passa por quatro saltos, e cada um espera pelo ficheiro inteiro antes de o enviar ao seguinte. Os serviços Node usam todos `multer.memoryStorage()`, por isso o ficheiro fica em RAM em cada serviço e ainda é copiado mais vezes.

Salto	Evidência no código	O que acontece ao ficheiro
nginx	`nginx/nginx.conf`, `client_max_body_size 1100m`, sem `proxy_request_buffering off`	O pedido inteiro é guardado em disco temporário antes de chegar ao apiGateway.
apiGateway	`apiGateway/routes/projects.js`, linhas 23–24 e rota `/:user/:project/video`	`memoryStorage` (1 cópia) e um `FormData` com `req.file.buffer` para o reenviar (2.ª cópia). O multer corre antes de `auth.checkToken`, por isso um pedido sem token também é lido inteiro.
projects	`projects/routes/projects.js`, linhas 72–73 e `addProjectMedia` (linha 1269); `projects/utils/video.js`, `probeVideoBuffer` (linha 84)	`memoryStorage`; o buffer é escrito num ficheiro temporário para o `ffprobe`; novo `FormData` para o armazenamento; `sha256` sobre o buffer. A validação de tamanho e duração só acontece depois de o ficheiro estar todo em memória.
imageStorageService	`route/upload.js` e `services/uploadImage.js`	`memoryStorage` e `s3.upload({Body: file.buffer})` para o SeaweedFS.

O frontend (`frontend/lib/projects.ts`, linha 473) envia o ficheiro com `api.post` sem `onUploadProgress`, por isso o utilizador não vê progresso durante um carregamento que pode demorar minutos.

Consequências:

- O pico de RAM cresce linearmente com o tamanho do ficheiro e com o número de carregamentos simultâneos.
- Três Premium a carregar 1 GB ao mesmo tempo podem esgotar a memória do anfitrião (OOM).
- A latência soma os tempos de todos os saltos, porque nenhum começa a enviar antes de receber tudo.

Medição exploratória. O script `medicoes/upload_rss.js` usa apenas Node 22. Simula um salto que faz o mesmo que o projects atual: buffer, cópia para `FormData`, ficheiro para o `ffprobe` e hash. Compara-o com um salto que faz streaming para disco e calcula o hash incrementalmente (Figura 1).

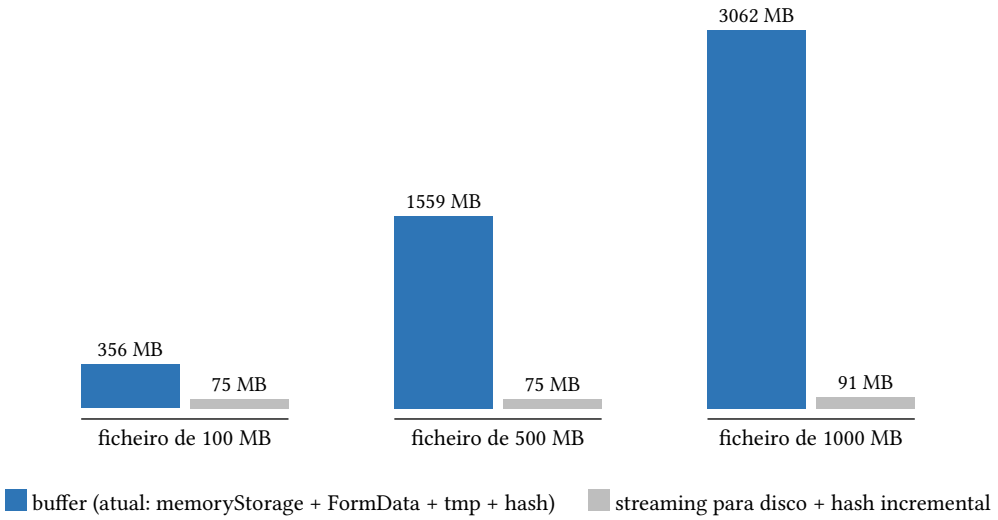


Figura 1: Pico de RSS de um salto (MB). Bufferizar custa cerca de 3× o ficheiro em RAM; o streaming fica perto de 90 MB. Fonte: `medicoes/upload_rss.js`, Node 22, 2 vCPU, 8 GB, 1 execução por ponto, 11 nov 2026.

O projects atual tem um salto deste tipo, e o apiGateway e o imageStorageService têm outros parecidos. Somando os três, um único vídeo de 1 GB pode ocupar entre 5 e 9 GB de RAM em simultâneo. O modo streaming também foi cerca de 2 vezes mais rápido por salto: 2,1 s contra 4,1 s para 1000 MB.

3 RNF-01 — Quantificação, método e refatoração proposta

Enunciado proposto. O carregamento de um vídeo deve usar memória constante, independente do tamanho do ficheiro. O carregamento de um vídeo Premium de 1 GB não pode levar nenhum serviço a ficar sem memória. O utilizador deve ver o progresso do carregamento.

Métrica	Escala	Método de medição	Valor base	Alvo	Mínimo aceitável
M1 Pico de RAM adicional por carregamento	MB acima do repouso, somando apiGateway, projects e imageStorageService	`docker stats` a cada 0,5 s durante o carregamento de 100 MB, 500 MB e 1 GB, com 5 repetições; reporta-se a mediana	A medir; previsão de 5–9 GB para 1 GB	≤ 300 MB, sem crescer com o tamanho	≤ 1 GB para um ficheiro de 1 GB
M2 Tempo de carregamento ponta a ponta	Segundos, do início do POST até ao 204	Script com `curl` em loopback, ficheiro de 1 GB, 5 repetições	A medir	≤ base − 30 %	≤ base (não piorar)
M3 Carregamentos Premium de 1 GB em simultâneo sem falhas	Número	1, 2, 4 e 8 em paralelo, num contentor com limite de 2 GB; conta-se quantos terminam com 204	A medir; previsão ≤ 1	4	2
M4 Rejeição de um ficheiro acima do limite	Bytes lidos antes do erro 413/400	Enviar 1,5 GB com o perfil free, cujo limite é 100 MB	~1,1 GB (o nginx lê até 1100m)	≤ limite do perfil + 1 MB	≤ 2 × limite do perfil

M5 Progresso visível	Sim/Não, e atualizações por segundo	Teste manual no browser	Não	Sim, ≥ 1 atualização/s	Sim

Origem dos valores.

- O limite de 1 GB vem de `VIDEO_PREMIUM_MAX_SIZE_MB: 1024` no `docker-compose.yaml`.
- Os 300 MB de M1 vêm da medição exploratória: cerca de 90 MB por salto em streaming, vezes três saltos.
- O alvo de 4 carregamentos simultâneos em M3 é uma decisão do grupo, que deve ser justificada no relatório (por exemplo, um pico esperado de Premium).

Alternativas.

Alternativa	Como funciona	Vantagens	Custos e riscos
A1 Streaming em todos os saltos (recomendada)	O nginx passa a usar `proxy_request_buffering off`. O apiGateway faz pipe do pedido para o projects, com o multer depois do token ou sem multer. O projects grava em disco com `busboy` / `diskStorage`, corre o `ffprobe` sobre o ficheiro, calcula o hash por stream e envia um stream ao armazenamento. O storage usa `s3.upload` com `Body` em stream, que faz multipart.	Memória constante; mantém a API e o contrato com o frontend; mudança localizada.	Ficheiros temporários têm de ser limpos; é preciso tratar pedidos abortados.
A2 Carregamento direto para o S3 com URL pré-assinado (multipart)	O frontend pede um URL ao projects, envia as partes diretamente ao SeaweedFS e confirma no fim; o projects valida depois.	O backend Node não toca no ficheiro; permite retomar um carregamento interrompido.	Muda o fluxo e o contrato da API; obriga a expor o S3 através do nginx; a validação fica para depois do carregamento. É mais arriscado em 9 dias.
A3 Carregamento por partes (estilo tus) através do gateway	O frontend envia partes de, por exemplo, 8 MB, e o servidor junta-as.	Permite retomar; memória limitada pelo tamanho da parte.	Mais código nos dois lados; aproxima-se de uma funcionalidade nova.

A recomendação é a **A1, acrescentando progresso no frontend** (`onUploadProgress` do axios, sem nova funcionalidade de negócio). Antes de ler o corpo do pedido, valida-se cedo o `Content-Length` contra o limite do perfil. A A2 fica no relatório como alternativa avaliada e rejeitada, com a justificação do custo e do risco.

4 RNF-02 — Diagnóstico da elasticidade do processamento de vídeo

A capacidade de processamento de vídeo é fixa: um contentor, um consumidor e um vídeo de cada vez. Nada aumenta quando a fila cresce, e nada diminui quando está vazia.

Aspeto	Evidência no código	Efeito
Uma única instância	`docker-compose.yaml`, serviço `video_tool` (linha 256): sem `deploy.replicas` e sem limites de CPU ou memória	Toda a carga de vídeo passa por um processo.
Um vídeo de cada vez	`Tools/video/video.py`: `basic_qos(prefetch_count=1)`, `BlockingConnection` e ack só no fim	O processamento é em série. O próprio comentário diz "parallelism comes from replicas", mas não há réplicas.
Pré-visualização e processamento na mesma fila	`projects/routes/projects.js`, linhas 1112–1121 (`preview: true`) e 1630–1639 (`preview: false`); fila única `video_queue` em `project_msg.js`	Uma pré-visualização de 3 s espera atrás de todos os vídeos completos que estejam à frente (bloqueio na cabeça da fila).

Legendas com Whisper no CPU	`Tools/video/subtitles.py`: `speech_model()` carrega o modelo `small` em cada trabalho, com `WHISPER_CPU_THREADS=2`	Um trabalho de legendas demora vários minutos para 10 min de vídeo e bloqueia todos os outros.
Volume partilhado	`image_data:/app/images` montado no projects e no `video_tool`; o projects descarrega os originais do S3 para o volume antes de publicar (linhas 1603–1620)	As réplicas têm de estar no mesmo anfitrião. Com Docker Compose isto é aceitável, mas deve ficar escrito como limitação.
Ligação com `heartbeat=0`	`video.py`, `_connect`	Se um worker ficar pendurado, o broker não o deteta; a mensagem fica presa até a ligação cair.

Medição exploratória. O script `medicoes/fila.py` simula a `video_queue` com 4 trabalhos completos e 1 pré-visualização no fim. Cada trabalho completo é uma recodificação de 20 s em 720p com `libx264 -preset veryfast`, as mesmas opções do worker. Correu numa máquina de 2 vCPU.

Consumidores	Duração de cada trabalho completo	Espera da pré-visualização	Tempo total
1 (atual)	3,6 s	14,7 s	15,3 s
2 na mesma máquina	7,1 s	14,2 s	14,8 s

Há duas conclusões. Primeiro, a pré-visualização espera sempre pela fila inteira, por isso um utilizador que pede 3 s de pré-visualização pode esperar minutos. Segundo, o `libx264` já usa todos os núcleos disponíveis. Mais consumidores só aumentam o débito se tiverem CPU próprio, ou seja réplicas com quota de CPU numa máquina com mais núcleos. Isto deve ser confirmado nos portáteis do grupo, que têm mais núcleos do que este ambiente.

5 RNF-02 — Quantificação, método e refatoração proposta

Enunciado proposto. O processamento de vídeo deve ajustar automaticamente o número de workers à carga, entre um mínimo e um máximo configuráveis. Uma pré-visualização não deve esperar por processamentos completos.

Métrica	Escala	Método de medição	Valor base	Alvo	Mínimo aceitável
E1 Tempo de expansão	Segundos, desde a fila ultrapassar o limiar até o novo worker consumir	Injetar 10 trabalhos de uma vez; registar timestamps do scaler e dos logs `[video] msg=`	Infinito (não escala)	≤ 30 s	≤ 60 s
E2 Tempo de contração	Segundos, desde a fila esvaziar até voltar ao mínimo	O mesmo ensaio, após o último trabalho	Não aplicável (fixo em 1)	≤ 120 s	≤ 300 s
E3 Débito com carga	Vídeos de 60 s processados por minuto	20 trabalhos, máximo de 4 réplicas com 1 CPU cada	A medir com 1 worker	≥ 3 × base	≥ 2 × base
E4 Espera da pré-visualização com a fila cheia	Segundos até começar, percentil 95	10 pré-visualizações durante o ensaio E3	A medir; exploratoriamente 14,7 s com só 4 trabalhos à frente	≤ 5 s	≤ 10 s
E5 Recursos em repouso	CPU e RAM dos workers de vídeo com a fila vazia	`docker stats` 5 min depois do fim do ensaio	1 worker sempre ligado	Igual à base (mínimo de 1)	≤ base + 1 worker

Origem dos valores.

- E3 e o máximo de 4 réplicas dependem dos núcleos do portátil de medição; o grupo deve registar esse hardware no relatório.
- O alvo de 5 s em E4 alinha com o tempo de resposta considerado aceitável nas pré-visualizações da Fase 1. Confirmar no relatório da PL1-G4.

Alternativas.

Alternativa	Como funciona	Vantagens	Custos e riscos
B1 Réplicas fixas (`deploy.replicas: N`)	Mais workers sempre ligados	Trivial	É escalabilidade, não elasticidade; desperdiça recursos em repouso; não resolve a pré-visualização.
B2 Autoscaler próprio (recomendada)	Um serviço pequeno lê a profundidade da `video_queue` na API de gestão do RabbitMQ e chama a API do Docker para ajustar as réplicas entre `MIN` e `MAX`, com tempo de arrefecimento	Elasticidade real, mensurável e reproduzível com Docker Compose	Precisa de `docker.sock` montado (efeito na segurança, a documentar). A regra de escala é código novo de infraestrutura, não uma funcionalidade.
B3 Kubernetes com HPA ou KEDA	Escala por métricas da fila	Solução padrão da indústria	Migrar todo o deploy em 9 dias não é realista; é difícil de reproduzir pelos docentes.
B4 Separar filas	`video_preview_queue` com worker dedicado (ou prioridade `x-max-priority`), e legendas numa fila própria	Elimina o bloqueio na cabeça da fila; o Whisper deixa de bloquear efeitos rápidos	Mais filas em `definitions.json`; o projects passa a escolher a fila.

A recomendação é **B2 + B4**:

- uma fila de pré-visualização com o seu próprio worker;
- a fila de processamento com autoscaler entre 1 e 4 réplicas, cada uma com limite de CPU e memória.

Três correções pequenas tornam o worker seguro para escalar:

- carregar o modelo Whisper uma vez por processo;
- ativar o heartbeat, com o processamento numa thread;
- fazer *graceful shutdown* quando o autoscaler remove uma réplica, sem ack, para a mensagem voltar à fila.

6 Efeitos colaterais, regressão e plano até 20 de novembro

Efeitos colaterais a medir e discutir no relatório:

- **Segurança.** Hoje o multer do apiGateway lê o ficheiro antes de validar o token; a A1 passa a validar antes, o que é uma melhoria. A B2 monta `docker.sock`, o que é um risco a limitar: um socket só de leitura via proxy, ou uma rede isolada.
- **Fiabilidade.** Os ficheiros temporários de carregamentos abortados têm de ser limpos. Ao remover uma réplica, um trabalho a meio tem de voltar à fila. Ambos são testados.
- **Disco.** O streaming troca RAM por disco temporário, por isso é preciso medir o espaço em `/tmp` no pico.
- **Manutenibilidade.** Surge mais um serviço (o autoscaler) e mais filas, que têm de ser documentados.
- **Custo em repouso.** Com o mínimo de 1 réplica fica igual ao atual. O worker de pré-visualização acrescenta 1 contentor.

Testes de regressão:

- Os testes existentes devem continuar a passar: `Tools/video/tests` (pytest) e `projects/test`.
- Novos testes do RNF-01:
  - um carregamento de 150 MB com o perfil free é rejeitado com o mesmo código de antes;
  - o hash `og_sha256` é igual ao calculado com o buffer;
  - um nome duplicado continua a dar 409/400;

  - um carregamento abortado não deixa ficheiros temporários.
- Novos testes do RNF-02:
  - a pré-visualização é publicada na fila nova;
  - a regra do autoscaler, como teste unitário da função `desired_replicas(depth, current)`;
  - um worker terminado a meio não faz ack e a mensagem é reentregue.

Plano (entrega a 20 nov; defesas de 23 a 27 nov):

Data	Tarefa	Feito
até 12 nov	Grupo decide os dois RNF, os alvos e o hardware de medição	x
13 nov	Medição de base no Docker (M1–M5 e E1–E5) com os scripts e dados guardados em `medicoes/`	❐
14 a 16 nov	Refatoração do RNF-01 (A1 e progresso no frontend), com os testes	❐
15 a 17 nov	Refatoração do RNF-02 (B4, depois B2 e as correções do worker), com os testes	❐
18 nov	Medição final nas mesmas condições e análise dos efeitos colaterais	❐
18 a 19 nov	Relatório: diagnóstico, quantificação, alternativas, uso de IA, resultados e commit de partida	❐
20 nov	Revisão final e entrega	❐

Perguntas em aberto para o grupo:

- Os alvos M3 (4 carregamentos simultâneos) e E3 (pelo menos 3 vezes o débito base) são aceitáveis?
- Que portátil será usado nas medições oficiais (núcleos e RAM)?